

# Teste Prático — Gestão de Escala de Plantões

## Contexto

No contexto de um segmento hospitalar, uma das funcionalidades solicitadas por clientes é o módulo de escala de plantões, que organiza a cobertura das unidades de internação ao longo da semana.

Você foi designado para desenvolver um **MVP** desse módulo.

Esse MVP será usado exclusivamente para caráter avaliativo e classificatório e não tem qualquer pretensão de uso ou aproveitamento em contexto real/produtivo.

---

## O que entregar

Uma aplicação fullstack com backend em **Java + Spring Boot** e frontend (que poderá ser VueJS ou Flutter, preferencialmente, mas será também aceito caso opte-se por outra tecnologia mais familiar ao candidato, como Angular, React ou puramente Javascript/CSS/HTML), integrados via API REST.

O código fonte será de propriedade do candidato e não será copiado ou usado pela empresa.

---

## Requisitos funcionais

Os requisitos abaixo devem ser encarados como linhas mestre para o desenvolvimento e muitos detalhes estão, propositalmente, ausentes, cabendo ao candidato tomar as decisões que julgar mais adequada para preencher essas lacunas. Um dos critérios de avaliação é a criatividade quanto às soluções e caminhos adotados na implementação.

### 1. Profissionais

- Cadastro com: nome, CRM/COREN, categoria ( MÉDICO | ENFERMEIRO | TÉCNICO ), carga horária semanal contratada (ex: 20h, 30h, 40h)
- Listagem com filtro por categoria

### 2. Escala

- Criação de um plantão vinculando: profissional, data e turno ( MANHÃ 07–13h | TARDE 13–19h | NOITE 19–07h` )
- Exclusão de um plantão agendado
- Listagem da escala por semana (filtro por data inicial)

### 3. Validações obrigatórias

- Um profissional não pode ter dois plantões no mesmo dia e turno
- A carga horária alocada na semana não pode ultrapassar a carga contratada (considere: manhã/tarde = 6h, noturno = 12h)

### 4. Visualização

- Grade semanal simples: linhas = profissionais, colunas = dias da semana, células = turno alocado (se houver)
- Destaque visual nos profissionais que atingiram o limite de horas na semana

---

## O que NÃO é necessário

- Autenticação/login
- Múltiplas unidades ou especialidades
- Geração automática de escala
- Cobertura mínima por turno

---

## Requisitos técnicos

- API REST com respostas e status HTTP adequados
- Banco de dados H2 (in-memory) ou PostgreSQL via Docker Compose — sua escolha
- Ao menos testes unitários cobrindo as duas regras de validação

---

## CrITÉrios de avaliação

| Dimensão  | O que será observado                                              |
|-----------|-------------------------------------------------------------------|
| Modelagem | Estrutura das entidades, relacionamentos, coerência com o domínio |

| Dimensão          | O que será observado                                                           |
|-------------------|--------------------------------------------------------------------------------|
| API               | Design dos endpoints, uso correto de verbos e status HTTP, tratamento de erros |
| Regras de negócio | Onde a lógica está implementada (service vs. controller vs. banco)             |
| Testes            | Cobertura das validações, clareza dos cenários                                 |
| Frontend          | Funcional e navegável — não se avalia design, mas clareza e usabilidade básica |
| Decisões          | README explicando escolhas técnicas relevantes                                 |
| Histórico git     | Granularidade e qualidade das mensagens de commit                              |

---

## Prazo e entrega

- **Prazo:** 4 dias corridos a partir do recebimento
- **Entrega:** repositório no GitHub ou GitLab (público ou com acesso compartilhado)
- **Após entrega:** sessão de 45 min de revisão de código ao vivo

---

## Dica ao candidato

O objetivo não é um sistema completo — é código que você possa defender. Prefira menos funcionalidades bem feitas a mais funcionalidades superficiais.